

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №26

Тема: Створення алгоритмів розв'язування рівнянь

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати чисельні методи розв'язування нелінійних рівнянь $f(x) = 0$ — методи половинного ділення (бісекції), простої ітерації та Ньютона (дотичних), навчитися відокремлювати корені, обирати метод і критерій зупинки та порівнювати швидкість збіжності методів.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), бібліотека matplotlib (для графіків).

Короткі теоретичні відомості

1. Постановка задачі. Потрібно знайти корінь x^* рівняння $f(x) = 0$, тобто значення, при якому $f(x) = 0$, із заданою точністю ε . Для більшості нелінійних рівнянь формули кореня не існує, тому застосовують ітераційні методи*: будують послідовність наближень $x_0, x_1, x_2, \dots$, що збігається до кореня.

2. Відокремлення кореня. Спочатку знаходять відрізок $[a, b]$, на якому лежить рівно один корінь. Достатня умова: f неперервна на $[a, b]$, $f(a) \cdot f(b) < 0$ (зміна знака) і f монотонна (f' зберігає знак). Для пошуку відрізка використовують табулювання функції (практична робота №23) або графік.

3. Метод половинного ділення (бісекції). Відрізок $[a, b]$ із $f(a) \cdot f(b) < 0$ ділять навпіл: $c = (a + b)/2$. Якщо $f(a) \cdot f(c) \leq 0$, корінь у $[a, c]$ ($b := c$), інакше — у $[c, b]$ ($a := c$). Процес повторюють, доки довжина відрізка $b - a$ не стане меншою за ε . Метод завжди збігається, довжина відрізка зменшується вдвічі на кожному кроці, тому для точності ε потрібно $n \geq \log_2((b - a)/\varepsilon)$ ітерацій. Збіжність лінійна, швидкість стала.

4. Метод простої ітерації. Рівняння перетворюють до вигляду $x = \varphi(x)$ і будують $x_{k+1} = \varphi(x_k)$. Достатня умова збіжності: $|\varphi'(x)| \leq q < 1$ на відрізку, що містить корінь і всі наближення. Чим менше q , тим швидша збіжність; похибка спадає приблизно як q^k . Критерій зупинки: $|x_{k+1} - x_k| < \varepsilon$. Для того самого рівняння можна обрати різні φ — одні дають збіжність, інші розбігаються.

5. Метод Ньютона (дотичних). $x_{k+1} = x_k - f(x_k)/f'(x_k)$. Геометрично: наступне наближення — точка перетину дотичної до графіка в точці x_k з віссю Ox . Збіжність квадратична: кількість правильних цифр приблизно подвоюється на кожній ітерації, але лише при достатньо хорошому початковому наближенні. Недоліки: потрібна похідна; при $f'(x_k) \approx 0$ метод ламається; для поганого x_0

може розбігатися або «стрибати» до іншого кореня. Початкове наближення обирають із умовою $f(x_0) \cdot f''(x_0) > 0$.

Метод	Збіжність	Що потрібно	Особливості
Бісекція	лінійна, $q = 1/2$	f неперервна, $f(a)f(b) < 0$	завжди збігається, повільний
Проста ітерація	лінійна, $q = \varphi' $	перетворення $x = \varphi(x)$, $ \varphi' < 1$	залежить від вибору φ
Ньютона	квадратична	$f'(x)$, добре x_0	дуже швидкий, але може розбігатись

6. Критерії зупинки: за довжиною відрізка ($b - a < \epsilon$), за різницею наближень ($|x_{k+1} - x_k| < \epsilon$), за нев'язкою ($|f(x_k)| < \epsilon$). Для надійності додають обмеження кількості ітерацій.

Хід роботи

Завдання 1. Відокремлення кореня та ручна бісекція

Знайти вручну корінь рівняння $f(x) = x^3 - x - 2 = 0$ методом половинного ділення з точністю $\epsilon = 0,1$.

Порядок виконання:

1. Табулюйте $f(x)$ для $x = 0, 1, 2$ та знайдіть відрізок, на кінцях якого функція має різні знаки; перевірте, що на ньому $f'(x) = 3x^2 - 1 > 0$ (корінь єдиний).
2. Оцініть за формулою $n \geq \log_2((b - a)/\epsilon)$ необхідну кількість ітерацій для $\epsilon = 0,1$.
3. Побудуйте таблицю бісекції зі стовпцями «к», «а», «b», «с = (a+b)/2», «f(c)», «новий відрізок»; виконайте кроки до $b - a < 0,1$.
4. Запишіть наближений корінь як середину останнього відрізка й оцініть похибку (півдовжина відрізка).
5. Перевірте знайдене значення, підставивши його у вихідне рівняння, та порівняйте кількість виконаних ітерацій із теоретичною оцінкою.

Завдання 2. Ручні ітерації: метод простої ітерації та метод Ньютона

Для рівняння $x^3 - x - 2 = 0$ виконати по 3 ітерації методами простої ітерації та Ньютона, порівняти швидкість наближення до кореня $x^* \approx 1,5214$.

Порядок виконання:

1. Перетворіть рівняння до виду $x = \varphi(x)$ двома способами: (а) $\varphi_1(x) = x^3 - 2$; (б) $\varphi_2(x) = \sqrt[3]{x + 2}$. Перевірте умову збіжності $|\varphi'(x)| < 1$ на відрізку $[1; 2]$ для обох варіантів ($\varphi_1' = 3x^2$, $\varphi_2' = 1/(3 \cdot (x+2)^{2/3})$).

- Виконайте по 3 ітерації для кожного варіанта з $x_0 = 1,5$; для збіжного варіанта запишіть значення x_k та різницю $|x_{k+1} - x_k|$; для розбіжного – покажіть, як наближення віддаляються від кореня.
- Запишіть формулу методу Ньютона для даного рівняння: $x_{k+1} = x_k - (x_k^3 - x_k - 2)/(3x_k^2 - 1)$.
- Виконайте 3 ітерації методу Ньютона з $x_0 = 2$ (виберіть x_0 за умовою $f(x_0) \cdot f''(x_0) > 0$, $f'' = 6x$).
- Складіть таблицю « k — наближення методом ітерацій (φ_2) — наближення методом Ньютона — похибка» для $k = 0 \dots 3$ і порівняйте швидкість.

Завдання 3. Програмна реалізація трьох методів

Реалізувати мовою Python методи бісекції, простої ітерації та Ньютона з підрахунком ітерацій; розв'язати рівняння $x^3 - x - 2 = 0$ та $\cos x = x$ з точністю $\epsilon = 10^{-8}$.

Порядок виконання:

- Напишіть функцію `bisection(f, a, b, eps)`, що перевіряє умову $f(a) \cdot f(b) < 0$ і повертає пару (корінь, кількість ітерацій); якщо умова не виконана — повертає `None`.
- Напишіть функцію `fixed_point(phi, x0, eps, limit=1000)`: ітерації $x_{k+1} = \varphi(x_k)$ до $|x_{k+1} - x_k| < \epsilon$; якщо за `limit` кроків збіжність не досягнута — повертає `None`.
- Напишіть функцію `newton(f, df, x0, eps, limit=100)` за формулою Ньютона з перевіркою $df(x) \neq 0$.
- Розв'яжіть $x^3 - x - 2 = 0$ усіма методами ($a = 1$, $b = 2$; $\varphi(x) = \sqrt[3]{x+2}$; $x_0 = 2$) і порівняйте кількість ітерацій; перевірте результати ручних обчислень із завдань 1, 2.
- Спробуйте варіант $\varphi_1(x) = x^3 - 2$ із завдання 2 й переконайтеся, що метод розбігається (повертає `None`).
- Розв'яжіть рівняння $\cos x = x$ ($f(x) = \cos x - x$ на $[0; 1]$) та виведіть значення нев'язки $f(x^*)$ для кожного методу.

Приклад реалізації мовою Python:

```
python
import math

def bisection(f, a, b, eps):
    if f(a) * f(b) > 0:
        return None
    k = 0
    while b - a >= eps:
        c = (a + b) / 2
        if f(a) * f(c) <= 0:
            b = c
        else:
            a = c
        k += 1
    return (a + b) / 2, k
```

```

def fixed_point(phi, x0, eps, limit=1000):
    x = x0
    for k in range(1, limit + 1):
        x_new = phi(x)
        if abs(x_new) > 1e100:
            return None
        if abs(x_new - x) < eps:
            return x_new, k
        x = x_new
    return None

def newton(f, df, x0, eps, limit=100):
    x = x0
    for k in range(1, limit + 1):
        d = df(x)
        if d == 0:
            return None
        x_new = x - f(x) / d
        if abs(x_new - x) < eps:
            return x_new, k
        x = x_new
    return None

f = lambda x: x ** 3 - x - 2
df = lambda x: 3 * x ** 2 - 1
eps = 1e-8
print("Рівняння  $x^3 - x - 2 = 0$  (точний корінь 1.5213797068...):")
print("    бісекція", bisection(f, 1, 2, eps))
print("    проста ітерація", fixed_point(lambda x: (x + 2) ** (1 / 3), 1.5, eps))
print("    Ньютон", newton(f, df, 2, eps))
print("     $\phi(x) = x^3 - 2$ ", fixed_point(lambda x: x ** 3 - 2, 1.5, eps))

g = lambda x: math.cos(x) - x
dg = lambda x: -math.sin(x) - 1
print("\nРівняння  $\cos x = x$ :")
results = {
    "бісекція": bisection(g, 0, 1, eps),
    "проста ітерація": fixed_point(math.cos, 0.5, eps),
    "Ньютон": newton(g, dg, 0.5, eps),
}
for name, res in results.items():
    x, k = res
    print(f"    {name:<16} x = {x:.10f}, ітерацій {k:>3}, |f(x)| = {abs(g(x)):.1e}")

```

Результат виконання програми:

```

Рівняння  $x^3 - x - 2 = 0$  (точний корінь 1.5213797068...):
    бісекція      (1.5213797055184841, 27)
    проста ітерація (1.5213797062335328, 9)
    Ньютон        (1.5213797068045676, 5)
     $\phi(x) = x^3 - 2$   None

Рівняння  $\cos x = x$ :
    бісекція      x = 0.7390851341, ітерацій 27, |f(x)| = 1.5e-09
    проста ітерація x = 0.7390851372, ітерацій 45, |f(x)| = 6.7e-09
    Ньютон        x = 0.7390851332, ітерацій 4, |f(x)| = 0.0e+00

```

Висновок за результатами: для рівняння $x^3 - x - 2 = 0$ усі методи знаходять корінь 1.5213797068, але кількість ітерацій суттєво відрізняється: бісекція – 27, проста ітерація — 9, метод Ньютона — лише 5. Бісекція потребує рівно $\log_2((b - a)/\epsilon)$ кроків незалежно від функції, проста ітерація збігається швидше завдяки

малій константі стиснення ($|\varphi'| \approx 0,14$), а метод Ньютона демонструє квадратичну збіжність. Варіант $\varphi(x) = x^3 - 2$ із завдання 2 не збігається ($|\varphi'(1,52)| = 3x^2 \approx 6,9 > 1$) – функція повертає None, що підтверджує: вибір перетворення $x = \varphi(x)$ критично важливий. Рівняння $\cos x = x$ має корінь 0.7390851332 .

Завдання 4. Дослідження швидкості збіжності та проблемних випадків

Дослідити зміну похибки за ітераціями для трьох методів, виявити проблемні випадки методу Ньютона та порівняти всі методи за кількістю обчислень функції.

Порядок виконання:

1. Для рівняння $x^3 - x - 2 = 0$ виконайте по 8 ітерацій кожного методу (бісекція, проста ітерація з φ_2 , Ньютон) і запишіть похибку $|x_k - x^*|$ на кожній ітерації; побудуйте графік залежності похибки від k у логарифмічному масштабі по осі ординат.
2. За графіком визначте для кожного методу характер збіжності (лінійна/квадратична) і оцініть для кожного кількість ітерацій, потрібних для похибки менше 10^{-12} .
3. Дослідіть чутливість методу Ньютона до x_0 : розв'яжіть $x^3 - 2x + 2 = 0$ з $x_0 = 0$ (заиклення: $0 \rightarrow 1 \rightarrow 0 \rightarrow \dots$); переконайтеся, що при $x_0 = -2$ метод знаходить корінь.
4. Знайдіть $\sqrt{2}$ як корінь рівняння $x^2 - 2 = 0$ методом Ньютона з $x_0 = 1$ (вавилонський метод $x_{k+1} = (x_k + 2/x_k)/2$) та запишіть кількість правильних цифр на кожному кроці.
5. Підсумуйте результати в таблиці «метод — тип збіжності — ітерацій до 10^{-8} — потрібна похідна — гарантована збіжність».

Приклад реалізації мовою Python:

python

```
import math
import matplotlib.pyplot as plt

root = 1.5213797068045676
f = lambda x: x ** 3 - x - 2
df = lambda x: 3 * x ** 2 - 1
phi = lambda x: (x + 2) ** (1 / 3)

errs_bis, errs_fix, errs_new = [], [], []
a, b = 1.0, 2.0
x_fix, x_new = 1.5, 2.0
for k in range(1, 9):
    c = (a + b) / 2
    if f(a) * f(c) <= 0:
        b = c
    else:
        a = c
    errs_bis.append(abs((a + b) / 2 - root))
    x_fix = phi(x_fix)
    errs_fix.append(abs(x_fix - root))
    x_new = x_new - f(x_new) / df(x_new)
    errs_new.append(abs(x_new - root))

print("k    бісекція        проста ітерація    Ньютон")
for k in range(8):
```

```

print(f"{k + 1}   {errs_bis[k]:>10.2e}   {errs_fix[k]:>14.2e}   {errs_new[k]:>12.2e}")

x = 0.0
path = [x]
for _ in range(5):
    x = x - (x ** 3 - 2 * x + 2) / (3 * x ** 2 - 2)
    path.append(x)
print("\nx^3 - 2x + 2 = 0, x0 = 0:", path)
print("x^3 - 2x + 2 = 0, x0 = -2:", newton(lambda t: t ** 3 - 2 * t + 2, lambda t: 3 *
t ** 2 - 2, -2, 1e-10))

x = 1.0
print("\nКвадратний корінь з 2 (Ньютон, x0 = 1):")
for k in range(1, 6):
    x = (x + 2 / x) / 2
    print(f"   k = {k}: {x:.15f}   похибка {abs(x - math.sqrt(2)):.2e}")

plt.figure(figsize=(7, 4.2))
ks = range(1, 9)
plt.semilogy(ks, errs_bis, "o-", label="бісекція")
plt.semilogy(ks, errs_fix, "s-", label="проста ітерація")
plt.semilogy(ks, [max(e, 1e-17) for e in errs_new], "^-", label="Ньютон")
plt.xlabel("номер ітерації k")
plt.ylabel("похибка |x_k - x*|")
plt.title("Швидкість збіжності методів розв'язування рівнянь")
plt.grid(alpha=0.4, which="both")
plt.legend()
plt.show()

```

Результат виконання програми:

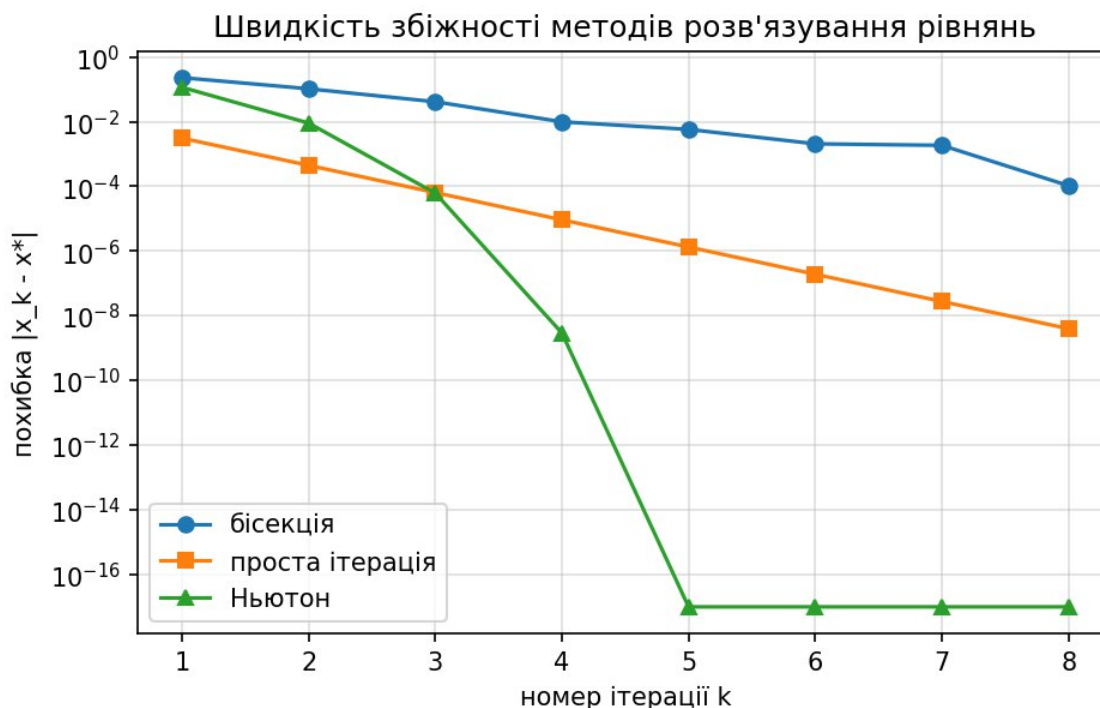
k	бісекція	проста ітерація	Ньютон
1	2.29e-01	3.09e-03	1.15e-01
2	1.04e-01	4.44e-04	9.01e-03
3	4.11e-02	6.40e-05	6.18e-05
4	9.87e-03	9.22e-06	2.93e-09
5	5.75e-03	1.33e-06	0.00e+00
6	2.06e-03	1.91e-07	0.00e+00
7	1.85e-03	2.75e-08	0.00e+00
8	1.05e-04	3.97e-09	0.00e+00

$x^3 - 2x + 2 = 0, x_0 = 0$: [0.0, 1.0, 0.0, 1.0, 0.0, 1.0]
 $x^3 - 2x + 2 = 0, x_0 = -2$: (-1.7692923542386314, 5)

Квадратний корінь з 2 (Ньютон, $x_0 = 1$):

k = 1:	1.5000000000000000	похибка 8.58e-02
k = 2:	1.4166666666666667	похибка 2.45e-03
k = 3:	1.414215686274510	похибка 2.12e-06
k = 4:	1.414213562374690	похибка 1.59e-12
k = 5:	1.414213562373095	похибка 2.22e-16

Графік порівняння:



Висновок за результатами: похибка бісекції зменшується приблизно вдвічі за ітерацію (пряма на графіку з малим нахилом), у простій ітерації – приблизно в 7 разів, а в методу Ньютона кількість правильних цифр подвоюється на кожному кроці: уже на 5-й ітерації похибка досягає межі точності чисел із плаваючою комою. Для рівняння $x^3 - 2x + 2 = 0$ при $x_0 = 0$ метод Ньютона зациклюється ($0 \rightarrow 1 \rightarrow 0 \rightarrow \dots$), оскільки початкове наближення вибране невдало, тоді як із $x_0 = -2$ корінь знаходиться швидко. Тому на практиці часто комбінують методи: бісекція дає надійне початкове наближення, а метод Ньютона його швидко уточнює.

Контрольні питання

1. У чому полягає задача розв'язування нелінійного рівняння? Що означає відокремлення кореня?
2. Опишіть алгоритм методу половинного ділення. Скільки ітерацій потрібно для досягнення точності ϵ ?
3. Сформулюйте умову збіжності методу простої ітерації. Як вибрати функцію $\phi(x)$?
4. Виведіть формулу методу Ньютона з геометричних міркувань. Який порядок збіжності цього методу?
5. Чому метод Ньютона може не збігатися або зациклюватися? Як цьому запобігти?
6. Які критерії зупинки ітераційних процесів ви знаєте? Чому потрібне обмеження кількості ітерацій?
7. Порівняйте методи бісекції, простої ітерації та Ньютона за надійністю та швидкістю.
8. Як вавилонський метод обчислення $\sqrt{a}$ пов'язаний із методом Ньютона?

Зміст звіту

1. Тема, мета роботи.
2. Таблиці ручної бісекції та ручних ітерацій (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік збіжності методів та підсумкова порівняльна таблиця.
5. Висновки.